Starknet 컴파일 특화 문서

개요

Starknet은 Ethereum 위에 구축된 Layer 2 스케일링 솔루션으로, Cairo 언어로 작성된 스마트 컨트랙트를 실행합니다. 일반적인 Cairo 프로그램과 달리 Starknet 컨트랙트는 특별한 컴파일 과정과 추가적인 메타데이터 생성이 필요합니다. 이 문서는 Cairo 컴파일러가 Starknet 컨트랙트를 어떻게 처리하는지 상세히 분석합니다.

Starknet 관련 Crate 구조

cairo-lang-starknet

Starknet 컴파일의 핵심 crate로, 다음과 같은 주요 컴포넌트를 포함합니다:

```
cairo-lang-starknet/
├── src/
| ├── lib.rs # 플러그인 스위트 정의
| ├── compile.rs # 컨트랙트 컴파일 메인 로직
| ├── contract.rs # 컨트랙트 선언 및 분석
| ├── abi.rs # ABI 생성 및 검증
| ├── plugin/ # Starknet 플러그인 시스템
| | ├── mod.rs # 메인 플러그인 구현
| | ├── consts.rs # 상수 및 속성 정의
| | ├── storage.rs # 스토리지 처리
```

```
| | ├── events.rs # 이벤트 처리
| | └── ...
| └── inline_macros/ # 인라인 매크로
└── test_data/ # 테스트 컨트랙트 예제
```

cairo-lang-starknet-classes

컨트랙트 클래스 정의와 직렬화를 담당하는 crate입니다:

```
cairo-lang-starknet-classes/
├── src/
| ├── contract_class.rs # Sierra 컨트랙트 클래스
| ├── casm_contract_class.rs # CASM 컨트랙트 클래스
| ├── abi.rs # ABI 데이터 구조
| ├── allowed_libfuncs.rs # 허용된 라이브러리 함수
| └── keccak.rs # 해시 계산
```

컨트랙트 컴파일 과정

1. 플러그인 시스템 초기화

Starknet 컴파일은 특별한 플러그인 스위트로 시작됩니다:

```
pub fn starknet_plugin_suite() -> PluginSuite {
let mut suite = PluginSuite::default();
```

```
suite
.add_plugin::<plugin::StarknetPlugin>()
.add_plugin::<plugin::storage_interfaces::StorageInterfacesPlugin>()
.add_inline_macro_plugin::<inline_macros::selector::SelectorMacro>()
.add_inline_macro_plugin::<inline_macros::get_dep_component::GetDepComponentMacr
.add_inline_macro_plugin::<inline_macros::get_dep_component::GetDepComponentMutM
.add_analyzer_plugin::<analyzer::ABIAnalyzer>()
.add_analyzer_plugin::<analyzer::StorageAnalyzer>();
suite
}
```

2. 컨트랙트 발견 및 검증

컴파일러는 다음 단계로 컨트랙트를 식별합니다:

1. **컨트랙트 모듈 탐색**: `#[starknet::contract]` 속성이 있는 모듈 검색
2. **컨트랙트 선언 생성**: `ContractDeclaration` 구조체로 컨트랙트 정보 저장
3. **단일성 검증**: 하나의 컨트랙트만 컴파일되도록 보장

```
pub struct ContractDeclaration<'db> {
/// 컨트랙트를 정의하는 모듈의 ID
pub submodule_id: SubmoduleId<'db>,
}
```

3. 엔트리 포인트 추출

Starknet 컨트랙트는 세 가지 유형의 엔트리 포인트를 가집니다:

External Functions (외부 함수)

- `#[external]` 속성 또는 `#[starknet::interface]` 구현
- 외부에서 호출 가능한 함수들
- 상태를 읽거나 수정할 수 있음

L1 Handler Functions (L1 핸들러)

- `#[l1_handler]` 속성
- Ethereum L1에서 오는 메시지 처리
- 첫 번째 매개변수는 반드시 `from_address`

Constructor (생성자)

- `#[constructor]` 속성
- 컨트랙트 배포 시 한 번만 실행
- 최대 하나만 존재 가능

```
pub struct SemanticEntryPoints<'db> {
pub external: Vec<Aliased<ConcreteFunctionWithBodyId<'db>>>,
pub l1_handler: Vec<Aliased<ConcreteFunctionWithBodyId<'db>>>,
pub constructor: Vec<Aliased<ConcreteFunctionWithBodyId<'db>>>,
}
```

ABI 처리와 인터페이스 생성

ABI 빌더 시스템

ABI(Application Binary Interface) 생성은 `AbiBuilder` 구조체가 담당합니다:

```
pub struct AbiBuilder<'db> {
db: &'db dyn SemanticGroup,
config: BuilderConfig,
abi_items: OrderedHashSet<Item>,
types: HashSet<TypeId<'db>>,
event_info: HashMap<TypeId<'db>, EventInfo>,
entry_points: HashMap<String, EntryPointInfo<'db>>,
ctor: Option<EntryPointInfo<'db>>,
errors: Vec<ABIError<'db>>,
}
```

ABI 항목 유형

1. **Functions**: 외부 함수들의 시그니처
2. **Constructor**: 생성자 정보
3. **L1Handler**: L1 핸들러 함수들
4. **Events**: 이벤트 정의
5. **Structs**: 사용된 구조체 타입들
6. **Enums**: 사용된 열거형 타입들
7. **Interfaces**: 구현된 인터페이스들
8. **Impls**: 임베디드 구현들

인터페이스 처리

Starknet 인터페이스는 `#[starknet::interface]` 속성으로 정의됩니다:

```
#[starknet::interface]
trait IMyContract<TContractState> {
fn  get_balance(self: @TContractState) -> u256;
fn  set_balance(ref  self: TContractState, amount: u256);
}
```

컴파일러는 이를 다음과 같이 처리합니다:

1. **디스패처 생성**: 인터페이스 호출을 위한 디스패처 구조체
2. **ABI 항목 추가**: 인터페이스의 모든 함수를 ABI에 포함
3. **타입 검증**: 매개변수와 반환 타입의 유효성 검사

스토리지 변수와 이벤트 처리

스토리지 시스템

Starknet 컨트랙트의 스토리지는 `Storage` 구조체로 정의됩니다:

```
#[storage]
struct Storage {
balance: u256,
owner: ContractAddress,
allowed: Map<ContractAddress, u256>,
}
```

컴파일러는 각 스토리지 변수에 대해 다음을 생성합니다:

1. **Getter/Setter 함수**: 스토리지 접근을 위한 함수들
2. **스토리지 주소 계산**: Pedersen 해시 기반 주소 매핑
3. **타입 검증**: 스토리지 가능한 타입인지 확인

이벤트 처리

이벤트는 `#[event]` 속성과 `Event` 트레이트로 정의됩니다:

```
#[event]
#[derive(Drop, starknet::Event)]
enum Event {
Transfer: Transfer,
Approval: Approval,
}

#[derive(Drop, starknet::Event)]
struct Transfer {
#[key]
from: ContractAddress,
#[key]
to: ContractAddress,
value: u256,
}
```

컴파일러의 이벤트 처리:

1. **이벤트 구조 분석**: 키와 데이터 필드 구분
2. **직렬화 코드 생성**: 이벤트 데이터의 직렬화/역직렬화
3. **셀렉터 계산**: 이벤트 식별을 위한 해시 값 생성
4. **ABI 포함**: 이벤트 정의를 ABI에 추가

컨트랙트 클래스 생성 과정

Sierra 컨트랙트 클래스

컴파일의 첫 번째 결과물은 Sierra 컨트랙트 클래스입니다:

```
pub struct ContractClass {
pub sierra_program: Vec<BigUintAsHex>,
pub sierra_program_debug_info: Option<sierra::debug_info::DebugInfo>,
pub contract_class_version: String,
pub entry_points_by_type: ContractEntryPoints,
pub abi: Option<Contract>,
}
```

주요 구성 요소:

1. **Sierra Program**: 컴파일된 Sierra 코드
2. **Entry Points**: 타입별로 분류된 엔트리 포인트들
3. **ABI**: 생성된 ABI 정보
4. **Debug Info**: 디버깅을 위한 메타데이터

CASM 컨트랙트 클래스

Sierra 클래스는 다시 CASM 클래스로 변환됩니다:

```
pub struct CasmContractClass {
pub prime: BigUintAsHex,
pub compiler_version: String,
pub bytecode: Vec<BigUintAsHex>,
pub bytecode_segment_lengths: NestedIntList,
pub hints: Vec<(usize, Vec<Hint>)>,
pub pythonic_hints: Option<Vec<(usize, Vec<PythonicHint>)>>,
pub entry_points_by_type: CasmContractEntryPoints,
}
```

변환 과정:

1. **Sierra → CASM 컴파일**: Sierra 명령어를 CASM으로 변환
2. **바이트코드 생성**: 실행 가능한 바이트코드 생성
3. **힌트 처리**: 증명 생성을 위한 힌트 정보
4. **세그멘테이션**: 바이트코드의 논리적 분할

컨트랙트 배포와 클래스 해시 계산

클래스 해시 계산

컨트랙트 클래스의 고유 식별자인 클래스 해시는 다음과 같이 계산됩니다:

1. **ABI 해시**: ABI 정보의 Poseidon 해시

2. **Sierra 프로그램 해시**: Sierra 코드의 해시

3. **엔트리 포인트 해시**: 각 타입별 엔트리 포인트 해시

4. **최종 해시**: 모든 구성 요소를 결합한 Poseidon 해시

```
pub fn compute_class_hash(contract_class: &ContractClass) -> Result<Felt252, .
let abi_hash = compute_abi_hash(&contract_class.abi)?;
let sierra_program_hash = compute_sierra_program_hash(&contract_class.sierra_pr
let entry_points_hash = compute_entry_points_hash(&contract_class.entry_points_
poseidon_hash_many(&[
CONTRACT_CLASS_VERSION_HASH,
entry_points_hash,
abi_hash,
sierra_program_hash,
])
}
```

배포 과정

1. **클래스 선언**: 컨트랙트 클래스를 네트워크에 등록

2. **인스턴스 배포**: 특정 주소에 컨트랙트 인스턴스 생성

3. **생성자 실행**: 배포 시 생성자 함수 호출

4. **상태 초기화**: 초기 스토리지 상태 설정

특별한 처리 과정

계정 컨트랙트 검증

`#[starknet::contract(account)]` 속성이 있는 컨트랙트는 추가 검증을 받습니다:

1. **필수 엔트리 포인트**: `__validate__`, `__execute__` 함수 존재 확인
2. **시그니처 검증**: 계정 컨트랙트 표준 준수 확인
3. **`__validate_deploy__` 검증**: 생성자와 매개변수 일치 확인

컴포넌트 시스템

Starknet은 재사용 가능한 컴포넌트 시스템을 지원합니다:

```
#[starknet::component]
mod ownable_component {
#[storage]
struct Storage {
owner: ContractAddress,
}
#[embeddable_as(Ownable)]
impl OwnableImpl<...> of IOwnableContract<ComponentState<TContractState>> {
// 구현...
}
}
```

컴파일러의 컴포넌트 처리:

1. **컴포넌트 분석**: `#[starknet::component]` 모듈 식별
2. **임베딩 처리**: `#[embeddable_as]` 구현 통합

3. **스토리지 병합**: 컴포넌트 스토리지를 메인 컨트랙트에 통합

4. **ABI 통합**: 컴포넌트 인터페이스를 컨트랙트 ABI에 포함

라이브러리 함수 검증

Starknet은 보안상의 이유로 허용된 라이브러리 함수만 사용할 수 있습니다:

```
pub fn validate_version_compatible(
contract_class: &ContractClass,
list_selector: ListSelector,
) -> Result<(), AllowedLibfuncsError> {
let allowed_libfuncs = lookup_allowed_libfuncs_list(list_selector)?;
let (_, _, sierra_program) = sierra_from_felt252s(&contract_class.sierra_program
for libfunc in &sierra_program.libfunc_declarations {
if !allowed_libfuncs.allowed_libfuncs.contains(&libfunc.long_id.generic_id) {
return Err(AllowedLibfuncsError::UnsupportedLibfunc { ... });
}
}
Ok(())
}
```

최적화 및 성능 고려사항

가스 효율성

1. **엔트리 포인트 비용**: 각 엔트리 포인트는 기본 10,000 가스 비용
2. **스토리지 접근 최적화**: 불필요한 스토리지 읽기/쓰기 최소화
3. **함수 인라이닝**: 작은 함수들의 인라인 처리

컴파일 시간 최적화

1. **증분 컴파일**: 변경된 부분만 재컴파일
2. **병렬 처리**: 독립적인 컴포넌트의 병렬 컴파일
3. **캐싱**: 중간 결과물의 캐싱 활용

에러 처리 및 진단

컴파일 에러 유형

1. **구문 에러**: 잘못된 Starknet 속성 사용
2. **의미 에러**: 타입 불일치, 누락된 함수 등
3. **ABI 에러**: 잘못된 인터페이스 정의
4. **스토리지 에러**: 지원되지 않는 스토리지 타입

진단 시스템

```
pub  enum ABIError<'db> {
EntrypointMustHaveSelf,
UnexpectedType,
MultipleConstructors(Source<'db>),
EventNotDerived(Source<'db>),
// ... 기타 에러들
```

```
}
```

컴파일러는 상세한 에러 메시지와 함께 소스 코드 위치를 제공합니다.

결론

Starknet 컴파일은 일반적인 Cairo 컴파일 과정에 다음과 같은 특별한 단계들을 추가합니다:

1. **플러그인 기반 처리**: Starknet 특화 문법과 속성 처리
2. **ABI 생성**: 외부 상호작용을 위한 인터페이스 정의
3. **클래스 생성**: 배포 가능한 컨트랙트 클래스 생성
4. **보안 검증**: 허용된 기능만 사용하도록 제한
5. **최적화**: 가스 효율성과 성능 최적화

이러한 과정을 통해 Cairo 코드는 Starknet 네트워크에서 실행 가능한 스마트 컨트랙트로 변환됩니다.